Data Structures 1 – Spring 2025 - Assignment A4

version 1.0

Rules:

- **Submission Deadline: June 13 at 23:59**. Late submissions will incur the following penalties:
  - **June 14:** 10-point deduction
  - **June 15:** 20-point deduction

  Submissions will not be accepted after June 15.
- The assignment should be completed in groups of two students (already assigned in Moodle).
- Students will be randomly selected to provide an oral explanation of their solution to the exercises.
- Questions regarding the exercise statements are welcome during office hours. However, teachers will not assess your solution.
- There is no guarantee that questions submitted via email will receive a response. Should you decide to submit questions by email anyway, you must address both professors: `herman.schinca@gtiit.edu.cn` and `hernan.melgratti@gtiit.edu.cn`. **Emails sent to only one professor will not be answered and no notification will be provided**.
- The code must be implemented in Java, compiled in VSCode, and pass the developed tests. This is the minimum requirement. Note that graders and professors may run additional tests. Furthermore, passing tests is not a sufficient condition for correctness. The quality of the submitted code will also be graded.
- Submit **one zip file containing the code for each exercise** (wet part) and **one PDF for all other exercises** (dry part).

THE DRY PART MUST BE PRODUCED USING A TEXT EDITOR (e.g., MS Word, LaTeX, or similar). Handwritten submissions are not permitted.

Plagiarism and Academic Misconduct:

In cases of copying or plagiarism, exams will be placed under review. If plagiarism is confirmed, the case will be reported to the GTIIT Discipline Committee. All group members will be held accountable for any submission that violates this policy. **THE USE OF AI TOOLS IS PERMITTED FOR THIS ASSIGNMENT**

Exercise 1: Minimum Spanning Tree [30 points]

When we ask two popular IA tool the following question:

```
Consider a graph with five vertices and the following edges:

A-B: 1

A-C: 1

A-E: 1

B-D: 3

B-E: 2

C-E: 1

C-D: 2

D-E: 2


a) List all the different minimum spanning trees.
b) Identify which of them can be obtained using Kruskal's algorithm.
```

we get the following contrasting answers:

```
Final Answer
a) All different minimum spanning trees:

There are five distinct minimum spanning trees with a total weight of 5.
They are:

{A-B, A-C, A-E, C-D}

{A-B, A-C, A-E, D-E}

{A-B, A-C, C-E, D-E}

{A-B, A-E, C-E, C-D}

{A-B, A-E, C-E, D-E}
```

```
b) MSTs obtainable using Kruskal's algorithm:

All five of the above minimum spanning trees can be obtained using
Kruskal's algorithm, depending on the order in which edges
of the same weight are selected when no cycle is formed.
```

and

```
Final Answers
a) All Minimum Spanning Trees (Weight = 5)
A-B, A-C, A-E, C-D

A-B, A-C, A-E, D-E

A-B, A-C, C-E, D-E

A-B, A-C, C-E, C-D

A-B, A-E, C-E, C-D

A-B, A-E, C-E, D-E

A-C, A-E, C-E, C-D

A-C, A-E, C-E, D-E

b) MSTs found by Kruskal's algorithm (default order):
Only MST 1: A-B, A-C, A-E, C-D
```

Tasks 1: Different spanning trees

What is the correct answer, if any, to the first question: "List all the different minimum spanning trees of the graph"? Justify your answer by explicitly constructing all distinct spanning trees.

Task 2: Kruskal

Determine whether any of the answers to the second question are correct. If so, identify which one(s).

Exercise 2: Dismounting sequence [20 points]

Given a directed graph representing the dependencies involved in mounting a complex object, where an edge `A → B` indicates that component **A must be installed before component B**, for example:

```
Motherboard → CPU
Motherboard → RAM
Motherboard → Power Supply
Power Supply → GPU
GPU → Case Cover
RAM → Case Cover
CPU → Case Cover
```

our goal is to produce a valid **dismounting sequence**—an order in which components can be safely removed without violating dependency constraints.

We are aimed at defining the following algorithm:

```
List<E> dismounting(Graph<E> graph)
```

that takes an unweighted DAG and produces a dismounting sequence.

Task 1: Algorithm

Describe an algorithm to compute a valid dismounting sequence based on the given dependency graph. Your solution should utilize algorithms covered in the course for graph traversal and **must not** simply reverse an installation sequence. You may define a suitable transformation to the graph if necessary.

Task 2: Complexity

Analyse the complexity of the proposed algorithm. Your analysis can rely on the complexity of the algorithm covered in the course.

Task 3: Implementation

Implement the described algorithm in Java. Include appropriate test cases.

Exercise 3: Train Route Planning [50 points]

Consider a directed graph where each vertex represents a train station and each directed edge represents a direct train connection from one station to another. A traveler aims to plan a trip that includes visits to two specific cities, regardless of the order in which they are visited. Among the possible routes, a route is considered more convenient if it requires fewer train changes. train changes: 換乘

Assume the traveler starts at station **A** and wishes to visit cities **X** and **Y**, in either order. The following two routes are possible:

- **Route 1**: A → B → X → C → Y (**3** train changes)
- **Route 2**: A → D → Y → X (**2** train changes)

In this scenario, Route 2 is considered more convenient, as it involves fewer train changes.

The objective is to define a method with the following signature:

`List<Station> convenientPath(Graph<Station>, Station start, Station target1, Station target2)`

This method receives:

- a directed graph representing the train map,
- a starting station,
- and two target stations to be visited (in any order).

The method must return a path that visits both target stations and is among those requiring the fewest possible train changes. If there is more than 1 possible answer, return any of them.

Task 1: Specification

Give the specification for the method `convenientPath`.

Note:

- A textual description will not be accepted as a solution.
- You may use the predicate `isPath(P, G)` without defining it to assert that a sequence `P` is a path in the graph `G`.
- Statements as the following are deem vague, and hence, incorrect: `Among all paths that satisfy conditions 1-3, P has the minimal number of edges (i.e., P.size() - 1 is minimized).`

Task 2: Algorithm

Describe an algorithm that satisfies the problem's requirements by composing existing graph algorithms studied in the course. No ad hoc graph traversal methods should be developed; instead, the solution should rely exclusively on the reuse of standards seen in the course.

Note that:

- Vague references to using a generic BFS or DFS algorithm will be considered insufficient or inaccurate.
- Unnecessary duplication of computations—even if the overall runtime complexity remains acceptable—will be regarded as an unsatisfactory solution.

Task 3: Correctness

Demonstrate the correctness of the proposed solution. The justification should be based on the established correctness of the algorithms used.

Task 4: Complexity

Analyze the worst-case time complexity of the solution. Your analysis can rely on the known complexity of the algorithms covered in the course.

Task 5: Java Implementation

Implement the method `convenientPath` in Java, using only the custom collections and data structures developed in the course. Your solution should not use Java Collections. Provide suitable unit tests for your implementation.